Universidade do Minho

Ano Letivo: 2025/26

Databases

SQL (continued)



Summary

1 **Data Control (DCL)**
- *User Management*
- *Granting and Removing Privileges*
- *Roles*

2 **Views, Indexes, Prepared Statements, Procedures**

Bibliography:

- Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. **(Chapter 18)**

- Belo, O., "Bases de Dados Relacionais: Implementação com MySQL", FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5. **(Chapter 2)**

Database

Sakila

Sakila is a sample database developed by MySQL and will be used in the examples of the next lessons.

It represents the operation of a movie rental store.

What's included?

Information about:

- Movies
- Customers
- Shops
- Payments
- Rentals

Tutorial:

https://dev.mysql.com/doc/sakila/en/sakila-introduction.html

Installation:

https://dev.mysql.com/doc/sakila/en/sakila-installation.html

The SQL Language

User Management

User management in a database system is done using Data Definition Language (DDL) statements.

For a user to be able to access the database, it is necessary to:

- have a profile created in the system
- have permissions (privileges) that define what they can or cannot do

User creation is performed using the CREATE USER statement.

To modify or delete users, the following are used, respectively:

- ALTER USER
- DROP USER

The SQL Language

User Management

- You can create users with local or remote access to a database
- Each user is identified by: name, origin (host) and password

Create local user

```
CREATE USER ‘cris’@’localhost’
IDENTIFIED BY ‘passCris’
```

The user can only access it from the system itself

Create remote user

```
CREATE USER 'gloria'@'11.0.1.1'
IDENTIFIED BY 'passGloria’;
```

User can access from the specified IP

Change Password

```
ALTER USER ‘cris’@’localhost’
IDENTIFIED BY ‘newCris’;
```

The SQL Language

User Consultation

The consultation of users allows obtaining information about access to the system.

List all users:

```
SELECT * FROM mysql.user;
```

Query name, host, and maximum number of connections:

```
SELECT User, Host, max_connections
FROM mysql.user
ORDER BY User;
```

Consultation of information relating to a set of users in the local system:

```
SELECT User, Host
FROM mysql.user
WHERE USER IN ('james','cris')
      AND Host = 'localhost';
```

These queries allow you to analyze who has access to the system and how.

The SQL Language

User Removal

Removing users allows access to the database system to be revoked.
It is used when a user no longer needs access or for security reasons.

DROP USER 'james'@'localhost';

Important aspects:

- The operation is irreversible
- The user loses all associated permissions
- It should be used carefully to avoid removing necessary access

The SQL Language

Assignment of Privileges

- Assigning privileges allows you to control user access to database objects
- Defines what each user can do (read, insert, update, delete, etc.)
- Most common privileges in SQL:
  - **ALL PRIVILEGES** – Grants all privileges to the user
  - **CREATE** – Allows you to create databases and tables
  - **DROP** – Allows you to delete databases and tables
  - **DELETE** – Allows you to remove records from a table
  - **INSERT** – Allows you to insert new records
  - **SELECT** – Allows you to query data
  - **UPDATE** – Allows you to change existing records

Assign privileges: GRANT
Remove privileges: REVOKE

The SQL Language

Assignment of Privileges

Privileges can be assigned at different levels:

- The entire system
- A database
- A table

Examples:

```
GRANT ALL PRIVILEGES ON *.* TO 'gloria'@'localhost';
```

All kinds of privileges, over all databases and all their tables

```
GRANT ALL PRIVILEGES ON sakila.* TO 'james'@'localhost';
```

All kinds of privileges, on the basis of Sakila data and all its tables

```
GRANT ALL PRIVILEGES ON sakila.customer TO 'james'@'localhost';
```

All kinds of privileges, on the basis of Sakila data only for the Customer table

Apply changes:

```
FLUSH PRIVILEGES;
```

forces MySQL to reload permissions

The SQL Language

Assignment of Privileges

Examples:

```
GRANT SELECT, INSERT ON sakila.customer TO 'aluno'@'localhost’;
```
allows the user to query data (SELECT) and insert new records (INSERT) into the customer table, but cannot change or delete data.

```
GRANT UPDATE (email) ON sakila.customer TO 'support'@'localhost’;
```
Allows the user to update only the Email column of the Customer table, without access to the other columns.

```
GRANT ALL PRIVILEGES ON sakila.* TO 'admin'@'localhost' WITH GRANT OPTION;
```
gives the user full control over the entire Sakila database and even allows them to assign permissions to other users.

```
GRANT EXECUTE ON PROCEDURE sakila.film_in_stock TO 'analista'@'localhost’;
```
Allows the user to run the Stored Procedure film_in_stock, but does not allow you to change its definition.

The SQL Language

Limitation of account actions

Queries per hour
Limits the number of queries executed

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_QUERIES_PER_HOUR 100;
```

Hourly modifications
Limits INSERT, UPDATE and DELETE

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_UPDATES_PER_HOUR 50;
```

Connections per hour
Limits the number of connections to the server

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_CONNECTIONS_PER_HOUR 20;
```

Simultaneous connections
Limits active connections at the same time

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_USER_CONNECTIONS 5;
```

The SQL Language

Roles

Roles (or groups) are sets of permissions that can be assigned to multiple users.

What are they for?
They allow you to manage permissions in a simpler and more organized way, avoiding assigning permissions individually to each user.

Advantages

- Easier to administer users
- Reduces permissions errors
- Allows you to reuse access settings

Creating a group is done using the CREATE ROLE statement.

The SQL Language

Roles

- **Create a role**

```
CREATE ROLE 'analista’;
```

- **Give role permissions**

```
GRANT SELECT ON sakila.* TO 'analista’;
```

- **Assign role to user**

```
GRANT 'analista' TO 'aluno'@'localhost’;
```

- **Enable role by default**

```
SET DEFAULT ROLE 'analista' TO 'aluno'@'localhost’;
```

- **Remove role**

```
DROP ROLE 'analista’;
```

The SQL Language

Privilege query

View a user's privileges

`SHOW GRANTS FOR 'aluno'@'localhost';` -> Shows all permissions assigned to the user

View current user

`SELECT CURRENT_USER();` -> Shows the user connected to the system

View assigned roles

`SHOW GRANTS FOR 'aluno'@'localhost';` -> It also shows the roles associated with the user

View active roles in session

`SELECT CURRENT_ROLE();` -> Shows which roles are currently active

The SQL Language

Views

- A view is a virtual table based on the result of a SQL query.
- It doesn't physically store data, it executes the query whenever it's called.

Basic Syntax

```
CREATE VIEW nome_da_view AS
SELECT coluna1, coluna2 FROM tabela WHERE condição;
```

To use:

```
SELECT * FROM nome_da_view;
```

A Linguagem SQL

Vistas

Para que servem?

- **Simplificar queries complexas** — esconde JOINs e lógica repetitiva
- **Segurança** — expõe apenas as colunas/linhas necessárias ao utilizador
- **Reutilização** — uma query definida uma vez, usada em vários sítios
- **Abstração** — o código da aplicação não depende da estrutura real das tabelas

The SQL Language

Views

Example 1 – List of films available for rent:

```
CREATE VIEW v_filmes_disponiveis AS
SELECT title, rental_rate, rating
FROM film
WHERE rental_duration <= 5;
```

Now just do:

```
SELECT * FROM v_filmes_disponiveis;
```

Example 2 - Active customers:

```
CREATE VIEW v_clientes_ativos AS
SELECT first_name, last_name, email
FROM customer  WHERE active = 1;

SELECT * FROM v_clientes_ativos;
```

The SQL Language

Views

Example

Summary of rentals by client:

```
CREATE VIEW v_resumo_clientes AS
SELECT c.customer_id, c.first_name, c.last_name,
     COUNT(r.rental_id) AS total_alugueres,
     SUM(p.amount) AS total_gasto
FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id
LEFT JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Now just do:

```
SELECT * FROM v_resumo_clientes WHERE total_gasto > 50;
```

The SQL Language

Views

Example

```
-- Change — add duration column
CREATE OR REPLACE VIEW v_resumo_clientes AS
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(r.rental_id) AS total_alugueres,
    SUM(p.amount)      AS total_gasto,
    AVG(f.length)      AS duracao_media_filmes
FROM customer c
LEFT JOIN rental r    ON c.customer_id   = r.customer_id
LEFT JOIN payment p   ON c.customer_id   = p.customer_id
LEFT JOIN inventory i ON r.inventory_id  = i.inventory_id
LEFT JOIN film f      ON i.film_id       = f.film_id
GROUP BY c.customer_id, c.first_name, c.last_name;

-- Delete
DROP VIEW v_resumo_clientes;
```

The SQL Language

Views

Views as a Security Mechanism

Views can be used to control and restrict access to data, preventing users from accessing the base tables directly.

Example – Hide sensitive customer data:

```
CREATE VIEW v_cliente_publico AS
SELECT customer_id, first_name, last_name, email FROM customer;
-- Internal columns are hidden

GRANT SELECT ON v_cliente_publico TO utilizador_atendimento;
REVOKE SELECT ON customer FROM utilizador_atendimento;
```

A Linguagem SQL

Vistas

Views como Mecanismo de Segurança

As views podem ser usadas para **controlar e restringir o acesso aos dados**, impedindo que os utilizadores acedam diretamente às tabelas base.

Exemplo — ocultar dados sensíveis de clientes:

```
CREATE VIEW v_cliente_publico AS SELECT customer_id, first_name, last_name,
email FROM customer; -- colunas internas ficam ocultas

GRANT SELECT ON v_cliente_publico TO utilizador_atendimento; REVOKE SELECT
ON customer FROM utilizador_atendimento;
```

The SQL Language

Indexes

- An index is an auxiliary structure that speeds up data retrieval in a table by avoiding the need to scan all records row by row.
  It is like a book index: instead of reading the entire book, you go directly to the right page.

Create Simple Index

```
CREATE INDEX idx_nome ON tabela (coluna);
```

Create Unique Index

```
CREATE UNIQUE INDEX idx_nome ON tabela (coluna);
```

Delete index

```
DROP INDEX idx_nome ON tabela;
```

The SQL Language

Indexes

What are they for?

- Speed up searches — queries with WHERE, JOIN, and ORDER BY become faster
- Ensure uniqueness — UNIQUE indexes prevent duplicate values
- Improve JOINs — columns used in JOIN operations benefit greatly from indexes

Disadvantage: they take up disk space and slow down INSERT, UPDATE, and DELETE operations because the index also needs to be updated.

The SQL Language

Indexes

Types

Type	Description
INDEX	Simple index, allows duplicates
UNIQUE	Does not allow duplicate values
PRIMARY KEY	Single index created automatically
FULLTEXT	Free text search
COMPOSITE	Multi-column index

The SQL Language

Indexes

Example

Search by customer last name → without index, runs through the entire table:

```
SELECT * FROM customer WHERE last_name = 'SMITH';
```

Create index to accelerate:

```
CREATE INDEX idx_customer_lastname ON customer (last_name);
```

Now the same query uses the index and is faster.

The SQL Language

Indexes

Composite Index

Used when frequently filtering for more than one column:

```
CREATE INDEX idx_customer_nome ON customer (last_name, first_name);

-- Benefits from the index
 SELECT * FROM customer WHERE last_name = 'SMITH' AND first_name = 'JOHN';

-- Also benefits (uses the first column of the index)
SELECT * FROM customer WHERE last_name = 'SMITH';

-- It does NOT benefit (skips the first column)
SELECT * FROM customer WHERE first_name = 'JOHN';
```

The SQL Language

Indexes

UNIQUE Index

```
CREATE UNIQUE INDEX idx_email ON customer (email);
```

Ensures that no two clients have the same email → any attempt to insert a duplicate results in an error.

View Existing Indexes

```
SHOW INDEX FROM customer;
```

The SQL Language

Indexes

When to use indexes?

Use when:

- The column frequently appears in WHERE, JOIN, or ORDER BY clauses
- The table has many records
- The column has high cardinality (many distinct values)

Avoid when:

- The table is small
- The column is updated very frequently
- The column has low cardinality (e.g., an active column with only 0 and 1)

The SQL Language

Prepared Statement

A prepared statement is an SQL query that is compiled once by the server and can be executed multiple times with different parameters.

It is like a form with blanks: the structure is always the same, only the values change.

```
-- 1. Prepare
PREPARE nome_stmt FROM 'SELECT * FROM tabela WHERE coluna = ?';

-- 2. Set parameter
SET @valor = 'exemplo';

-- 3. Run
EXECUTE nome_stmt USING @valor;

-- 4. Liberate
DEALLOCATE PREPARE nome_stmt;
```

The SQL Language

Prepared Statement

What are they used for?

Performance — the query is compiled/parsed only once, even when executed multiple times
Security — they protect against SQL injection by separating SQL code from data
Reusability — the same statement can be executed with different parameters

Example:

```
PREPARE stmt_aluguer FROM
    'SELECT first_name, last_name, email
     FROM customer
     WHERE store_id = ? AND active = ?';

SET @loja = 1;
SET @ativo = 1;
EXECUTE stmt_aluguer USING @loja, @ativo;

DEALLOCATE PREPARE stmt_aluguer;
```

The SQL Language

Stored Procedures

A stored procedure is a block of SQL code stored on the server with a name, which can be called whenever needed.
It is like a function in a programming language: it is defined once and can be called multiple times.

Sintaxe

```
DELIMITER $$

CREATE PROCEDURE nome_procedure()
BEGIN
    -- SQL code here
END$$

DELIMITER ;
```

To call:

```
CALL nome_procedure();
```

The SQL Language

Stored Procedures

What are they used for?

- **Reusability** — logic is defined once and called in multiple places
- **Performance** — compiled and stored on the server
- **Security** — users can call the procedure without directly accessing the tables
- **Maintenance** — changes are made in a single place

The SQL Language

Stored Procedures

Parameters: IN, OUT, INOUT

Type	Description
IN	Input value to the procedure (default)
OUT	Value returned by the procedure
INOUT	Input and return value

```
CREATE PROCEDURE nome(IN param1 INT, OUT param2 VARCHAR(50))
```

The SQL Language

Stored Procedures

Examples

List movies by rating:

```
DELIMITER $$
CREATE PROCEDURE sp_filmes_por_rating(
     IN p_rating VARCHAR(10)
)
BEGIN
     SELECT title, rental_rate, length
     FROM film WHERE rating = p_rating;
END$$
DELIMITER ;

CALL sp_filmes_por_rating('PG’);
CALL sp_filmes_por_rating('R');
```

The SQL Language

Stored Procedures

Examples

Count a customer's total rentals:

```
DELIMITER $$

CREATE PROCEDURE sp_total_alugueres(
     IN p_customer_id        INT,
     OUT p_total             INT
)
BEGIN
     SELECT COUNT(*) INTO p_total
     FROM rental WHERE customer_id = p_customer_id;
END$$

DELIMITER ;

CALL sp_total_alugueres(1, @total);
SELECT @total;
```

The SQL Language

Stored Procedures

Examples

Classify customer using conditions:

```
CALL sp_classificar_cliente(1);
```

```
DELIMITER $$

CREATE PROCEDURE sp_classificar_cliente(IN p_customer_id INT)
BEGIN
    DECLARE v_total INT;

    SELECT COUNT(*) INTO v_total
    FROM rental
    WHERE customer_id = p_customer_id;

    IF v_total > 30 THEN
        SELECT 'Cliente VIP' AS classificacao;
    ELSEIF v_total > 10 THEN
        SELECT 'Cliente Regular' AS classificacao;
    ELSE
        SELECT 'Cliente Ocasional' AS classificacao;
    END IF;
END$$

DELIMITER ;
```

The SQL Language

Comparison

	VIEW	PREPARED STATEMENT	STORED PROCEDURE
Definition	Stored query	Precompiled query	Program stored in the database
Purpose	Simplify queries	Improve performance and security	Automate complex logic
Reusability	Yes	Yes (temporary)	Yes (persistent)
Logic	Does not support logic	Very limited	Supports logic (IF, LOOP, etc.)
Parameters	No	Yes	Yes
Performance	Medium	High	High
Security	Access control	Prevents SQL Injection	Advanced control
Storage	Permanent	Temporary (session)	Permanent
Execution	Automatic on SELECT	EXECUTE after PREPARE	CALL procedure_name()
Typical Use	Simple queries	Repeated queries	Complex processes

Universidade do Minho

Ano Letivo: 2025/26

Databases

SQL (continued)



António Abelha | abelha@di.uminho.pt